Searching Algorithms

Lecture 30

CS 1001

Kirk Duran

Today

- Define an **algorithm** as a precise procedure for solving a computational problem.
- State the search problem independently of any particular Python implementation.
- Implement and trace **linear search** and reason about how much work it performs.
- Search lists of objects by examining selected attributes rather than comparing whole objects.
- Search nested collections such as a list of courses, each containing a list of students.
- Use a small predicate function or `lambda` to separate a search procedure from its matching rule.
- Explain how **binary search** uses ordered data to eliminate large portions of the search space.

Key idea

A search algorithm specifies *where to look next* and *when enough information has been found to stop*.

The Boundary for This Lecture

Today

- algorithmic problem statements
- pseudocode
- linear search
- searching object attributes
- nested searches
- simple predicate functions and `lambda`
- binary search
- best- and worst-case work

Not today

- sorting algorithms
- formal correctness proofs
- trees or graph search
- hash-table implementation
- recursive search algorithms
- advanced asymptotic analysis

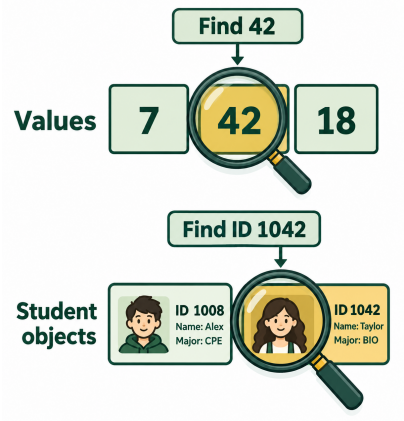
Part 1: From Problems to Algorithms

Searching Is a Computational Problem

Definition

A **search problem** asks whether some desired value or condition occurs within a collection and, when useful, where or what the matching item is.

- Is the number 42 in this list?
- At what index does `"Ada"` occur?
- Which student has ID 1042?
- Does any course contain a student with GPA above 3.8?



Definition: Algorithm

Definition

An **algorithm** is a finite, well-defined sequence of steps for solving a problem or performing a computation.

- **Well-defined:** each step has an unambiguous meaning.
- **Finite:** execution eventually terminates for inputs within the algorithm's intended domain.
- **Effective:** each step can actually be carried out.
- **General:** the procedure describes a family of inputs, not one memorized example.

Key idea

Python code is one way to express an algorithm; the algorithm itself is the underlying procedure.

One Problem Can Have More Than One Algorithm

Suppose we need to determine whether 37 appears in a collection.

Strategy A

Start at the beginning and examine each element until the target is found or the collection ends.

Linear Search

Strategy B

If the values are ordered, examine the middle and repeatedly discard the half that cannot contain the target.

Binary Search

Key idea

The representation and organization of the data determine which algorithms are available.

Pseudocode Separates the Procedure from Python Syntax

Linear-search pseudocode

```
LINEAR-SEARCH(values, target)
   for each position i from left to
right
      if values[i] equals target
         return i
   return NOT-FOUND
```

- Pseudocode communicates the computational structure.
- It intentionally omits language-specific punctuation and details.
- We can translate the same procedure into Python, Java, C++, or another language.

Correctness, Work, and the Search Contract

Before efficiency

A useful search must first be:

- **correct**: returned answers are right;
- **complete**: a match is found when one exists;
- **terminating**: the procedure eventually stops.

What should the function return?

- existence → `True`/`False`
- location → index
- matching item → value/object
- no match → `-1` or `None`

Key idea

Correctness comes first; then we ask how the work grows as the collection grows.

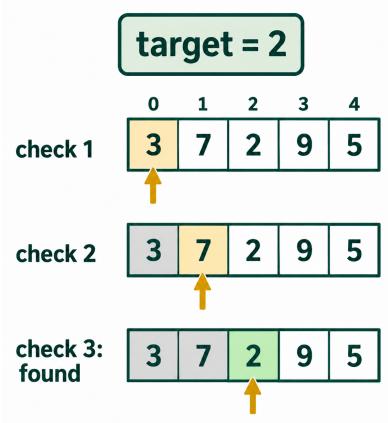
Part 2: Linear Search

Linear Search Examines Candidates in Sequence

Definition

Linear search examines elements one at a time until a match is found or every candidate has been checked.

- It does not require the data to be ordered.
- It can stop immediately after finding the desired item.
- If no match exists, every element must be ruled out.



Linear Search in Python

Return the index of the first match

```
def linear_search(values:  list[int], target:  int) -> int:
   for index, value in enumerate(values):
      if value == target:
         return index
   return -1
```

- Each iteration considers exactly one candidate.
- `return index` terminates the search as soon as the first match is known.
- Reaching `return -1` means the loop exhausted every candidate.

Trace: The Target Position Changes the Work

call	first match	checks	result
`linear_search([3,7,2,9,5], 7)`	index 1	2	1
`linear_search([3,7,2,9,5], 5)`	index 4	5	4
`linear_search([3,7,2,9,5], 8)`	absent	5	-1

- A match near the beginning lets the function return early.
- A match near the end requires more comparisons.
- To conclude “not found” in unsorted data, every candidate must be ruled out.

Best Case, Worst Case, and Linear Growth

For a list containing n elements:

Best case

The first element matches: approximately one comparison.

Worst case

The target is last or absent: approximately n comparisons.

Key idea

Worst-case linear search grows as $O(n)$: doubling the number of candidates can approximately double the required checks.

Linear Search Is More General Than Equality

The search procedure can test any condition that identifies the desired item.

Examples of matching rules

- `value == 42`
- `name == "Ada"`
- `student.student_id == 1042`
- `student.gpa >= 3.8`
- `course.code == "CS1001"`

Key idea

Searching means examining candidates until one satisfies the condition that defines a match.

Part 3: Searching Collections of Objects

Objects Change What We Compare

Student class

```
class Student:
   def __init__(self, student_id:  int, name:  str, gpa:  float):
      self.student_id = student_id
      self.name = name
      self.gpa = gpa
```

- A list can store references to `Student` instances.
- The target may not be an entire object.
- Usually we search by one meaningful attribute or condition.

Search a List of Objects by an Attribute

Return the matching student

```
def find_student(students:  list[Student], target_id:  int):
   for student in students:
      if student.student_id == target_id:
         return student
   return None
```

Key idea

The algorithm is still linear search. Only the matching rule changed from comparing primitive values to inspecting an object's attribute.

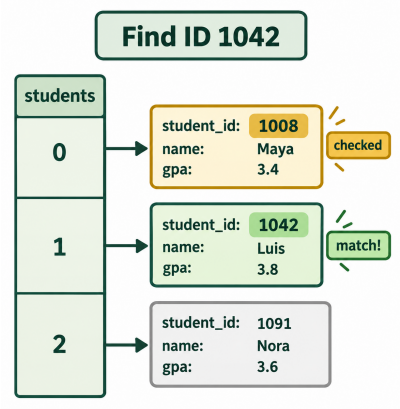
Trace: Object Search

Students

```
[Student(1008, "Maya", 3.4),
 Student(1042, "Luis", 3.8),
 Student(1091, "Nora", 3.6)]
```

Search for ID 1042:

1. inspect the object at position 0;
2. read `student.student_id`;
3. no match, advance;
4. inspect position 1;
5. matching attribute found, return that object.



What Should an Object Search Return?

Suppose a matching `Student` is found.

Return	Useful when the caller needs...
`True`	only whether a match exists
index	the object's position in the list
`Student` object	access to the matching student's state or methods
`None`	a clear "no matching object" result

Key idea

Returning the matching object is often more useful than returning its index because the caller can immediately access its attributes and methods.

From Specific Searches to a General Predicate

Exact IDs are only one matching rule. A search can instead receive a function that answers whether one candidate matches.

Reusable search

```
def find_first(items, matches):
   for item in items:
      if matches(item):
         return item
   return None
```

- `find_first` controls **how** candidates are examined.
- `matches` determines **which** candidates qualify.

A Small Use for `lambda`

A `lambda` expression creates a compact function value for a one-use matching rule.

Same search procedure, different predicates

```
find_first(students, lambda s:  s.student_id == 1042)
find_first(students, lambda s:  s.name == "Maya")
find_first(students, lambda s:  s.gpa >= 3.8)
```

Key idea

The `lambda` is not a new search algorithm. It only supplies the condition used by the existing linear search.

Part 4: Searching Nested Collections

Objects Can Contain Collections of Other Objects

Course class

```
class Course:
   def __init__(self, code:  str, students:  list[Student]):
      self.code = code
      self.students = students
```

A program might then store:

Outer collection

```
courses = [course1, course2, course3]
```

`list[Course]` → each `Course` → `list[Student]`

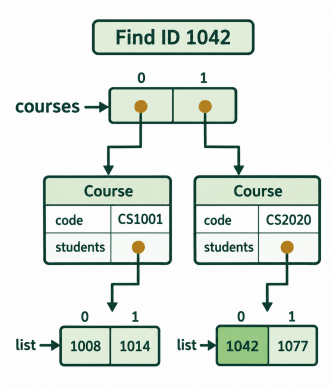
The Search Must Follow the Shape of the Data

To search every student across every course:

1. examine each `Course` in the outer list;
2. enter that course's `students` list;
3. examine each `Student`;
4. stop as soon as the matching student is found.

Key idea

Nested structure naturally produces nested traversal.



Student instances are labeled by ID; the list slots store references.

Search a List of Objects with Nested Lists of Objects

Return the first matching student

```
def find_student_in_courses(courses:  list[Course], target_id:  int):
   for course in courses:
      for student in course.students:
         if student.student_id == target_id:
            return student
   return None
```

- The outer loop moves among course objects.
- The inner loop moves among student objects belonging to the current course.
- A successful inner comparison terminates the entire function.

Trace the Nested Search — and Preserve Context

Suppose `CS1001` contains IDs `[1008, 1014]` and `CS2020` contains `[1042, 1077]`.

course	student ID	result
CS1001	1008	continue
CS1001	1014	continue
CS2020	1042	match

Returning `student` gives the matching object. Returning `(course, student)` preserves both the match and the container in which it was found.

Key idea

Search design includes deciding what context should accompany a successful match.

How Much Work Does the Nested Search Perform?

Suppose the courses contain 4, 6, and 5 students.

- A worst-case search can inspect all `4 + 6 + 5 = 15` students.
- Count **all student entries** plus visits to the course containers.
- The nested loops do not automatically mean the algorithm is "quadratic."

General rule

Total worst-case work is $O(C + S)$: C is the number of courses and S is the total number of student entries. Even an empty course requires an outer-loop visit.

A Common Nested-Search Bug: Returning Too Early

Incorrect

```
for course in courses:
    for student in course.students:
        if student.student_id == target_id:
            return student
        return None
```

Problem

The first nonmatching student immediately returns `None`. The function never examines later candidates.

Key idea

"Not found" is justified only after every relevant candidate has been ruled out.

Part 5: Binary Search

Linear Search Does Not Use Ordering Information

Consider an ordered list:

```
[3, 7, 12, 18, 24, 31, 42, 50, 61]
```

Searching for 42 linearly still begins at 3, then 7, then 12, and so on.

Observation

Because the values are ordered, comparing the target with one carefully chosen element can rule out many positions at once.

Binary Search Requires Ordered Data

Definition

Binary search repeatedly examines the middle of an ordered search interval and discards the half that cannot contain the target.

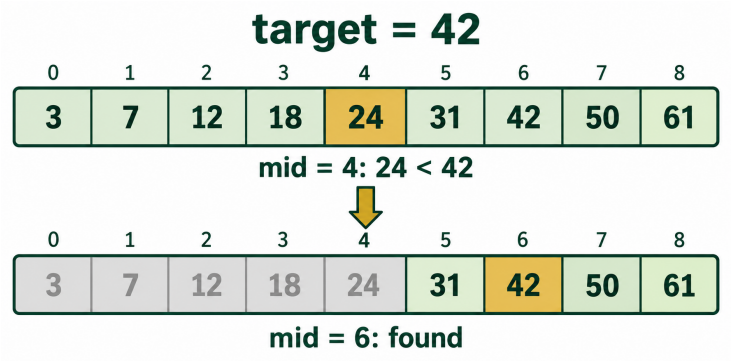
- If the middle value equals the target, stop.
- If the target is smaller, continue in the left half.
- If the target is larger, continue in the right half.

Precondition

Binary search depends on the collection being ordered in ascending order by the property being compared.

Binary Search Shrinks the Search Interval

Search for 42: compare with 24, keep indices 5–8, then inspect index 6.



Binary Search Maintains a Shrinking Interval

Precondition: `values` is sorted in ascending order.

Implementation

```
def binary_search(values:  list[int], target:  int) -> int:
   left = 0
   right = len(values) - 1
   while left <= right:
      mid = (left + right) // 2
      if values[mid] == target:  return mid
      elif values[mid] < target:  left = mid + 1
      else:  right = mid - 1
   return -1
```

Each comparison finds a match or shrinks `left ... right`. With duplicate values, this version may return any matching index.

Trace: Binary Search for 31

Search `[3, 7, 12, 18, 24, 31, 42, 50, 61]`.

left	right	mid	value	action
0	8	4	24	move `left` to 5
5	8	6	42	move `right` to 5
5	5	5	31	return 5

Only three elements were inspected.

Why Binary Search Is Faster on Large Ordered Data

Each unsuccessful comparison removes approximately half of the remaining candidates.

number of candidates	about how many halvings?
8	3
16	4
1,024	10
1,048,576	20

Key idea

For $n \geq 1$ candidates, this implementation inspects at most $\lfloor \log_2 n \rfloor + 1$ middle elements. Halvings and inspected elements differ by up to one; growth is $O(\log n)$.

Binary Search Fails Without the Ordering Precondition

Consider:

```
[18, 3, 42, 7, 31, 12, 24]
```

If the middle value is 7 and the target is 31, we cannot conclude that 31 must lie to the right.

Why not?

Values smaller and larger than 7 appear on both sides. The comparison does not identify a half that can safely be discarded.

Key idea

Binary search is powerful because ordering turns one comparison into information about many positions.

Binary Search Can Search Objects — by the Ordering Attribute

Suppose the list is ordered by `student_id`.

Middle comparison

```
student = students[mid]
if student.student_id == target_id:  ...
elif student.student_id < target_id:  ...
```

- We compare the attribute that defines the ordering, not the entire object.
- A list ordered by ID supports binary search by ID.
- That same list is *not* automatically suitable for binary search by GPA or name.

Key idea

The ordering and the search comparison must refer to the same property.

Linear Search vs. Binary Search

	Linear Search	**Binary Search**
Requires ordered data?	No	Yes
Candidate examined next	next element	middle of interval
Worst-case growth	$O(n)$	$O(\log n)$
Works naturally on arbitrary predicates?	Yes	only with compatible ordering
Simple nested object search?	Yes	usually not directly

Representation Determines the Available Search

representation	natural search approach
unordered list	linear search
ordered list	linear or binary search
dictionary by key	dictionary key lookup
nested object collections	traversal following the nested structure

Make assumptions visible

A binary-search function should clearly communicate its precondition: the collection must already be ordered by the property being compared.

Key idea

Choosing a data representation and choosing an algorithm are connected design decisions.

Integrated Trace: Which Search Is Appropriate?

For each task, choose the most natural search strategy.

1. Find the first student with GPA above `3.8` in an unsorted list.
2. Find student ID `1042` in a list already ordered by student ID.
3. Find student ID `1042` somewhere inside a list of course objects, each containing its own student list.
4. Retrieve the value associated with key `"Ada"` from a dictionary.

Reason from structure

Ask what organization the data already provides and what condition defines a match.

Takeaways

1. An algorithm is a precise, finite procedure for solving a computational problem.
2. Linear search checks candidates sequentially and works even when the data is unordered.
3. Searching objects usually means examining a selected attribute or predicate.
4. Nested object structures require traversal that follows the nesting of the data.
5. A `lambda` can compactly provide a one-use matching rule to a reusable search function.
6. Binary search repeatedly discards half of an **ordered** search interval.
7. Linear search has worst-case growth $O(n)$; binary search has worst-case growth $O(\log n)$.
8. Data organization determines which search strategies are valid and useful.